WEB DEVELOPMENT ASSIGNMENT

60-Second Countdown Timer Project Report

A self-contained web utility built with HTML, CSS, and JavaScript

Prepared For: Course Instructor / Presentation

Created by: Student

Date: August 2026

1. Executive Summary

This project showcases a responsive, interactive, and self-contained 60-second countdown timer webpage. The entire layout is designed to comply with a basic course assignment, keeping logic highly legible and codebase complexity minimal. By combining semantic HTML elements, clean modern CSS styling, and standard JavaScript timing functions, the application achieves a visual presentation that runs out-of-the-box on any modern web browser.

2. Key Design & Technical Features

- **Self-Contained File:**

All markup, CSS rules, and JavaScript logic are saved in one file, index.html. This removes loading errors and external dependency issues during student project submissions.

- **Interactive Controls:**

Three control actions: Start begins the 1000ms decrement interval; Pause clears the interval to halt time; Reset restores default state values.

- **Prominent CSS Styles:**

Styled using CSS Flexbox for centering, smooth button transitions, customized hover states, and standard color styling that adjusts automatically to different devices.

- **Font Stability:**

The timer uses a monospace font so the layout doesn't shift or stutter when swapping between wider digits (e.g. '0') and narrower ones (e.g. '1').

3. Implementation Source Code (index.html)

Below is the complete verbatim source code of the webpage implementation:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple 60-Second Countdown Timer</title>
    <style>
        /* General page layout */
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f3f4f6;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }

        /* Card layout for the timer */
        .timer-card {
            background-color: #ffffff;
            padding: 40px;
            border-radius: 16px;
            box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
            text-align: center;
            width: 320px;
        }

        /* Title style */
        h1 {
            margin: 0 0 10px 0;
            color: #1f2937;
            font-size: 24px;
        }

        p {
            margin: 0 0 20px 0;
            color: #6b7280;
            font-size: 14px;
        }

        /* Prominent countdown display */
        .countdown-display {
            font-size: 96px;
            font-weight: bold;
            color: #2563eb;
            margin: 20px 0;
            font-family: monospace; /* Keeps characters aligned */
            transition: color 0.3s ease;
        }

        /* Layout for controls */
        .controls {
            display: flex;
            gap: 12px;
            justify-content: center;
        }
```

```
        /* Button styles */
        button {
            padding: 12px 20px;
            font-size: 16px;
            font-weight: 600;
            border: none;
            border-radius: 8px;
            cursor: pointer;
            transition: all 0.2s ease;
            flex: 1;
        }

        .btn-start {
            background-color: #10b981;
            color: #ffffff;
        }

        .btn-start:hover {
            background-color: #059669;
        }

        .btn-pause {
            background-color: #f59e0b;
            color: #ffffff;
        }

        .btn-pause:hover {
            background-color: #d97706;
        }

        .btn-reset {
            background-color: #ef4444;
            color: #ffffff;
        }

        .btn-reset:hover {
            background-color: #dc2626;
        }

        /* Disabled state styling */
        button:disabled {
            background-color: #e5e7eb;
            color: #9ca3af;
            cursor: not-allowed;
        }
    </style>
</head>
<body>

    <div class="timer-card">
        <h1>Countdown Timer</h1>
        <p>Basic 60-Second Countdown</p>

        <!-- Large timer output -->
        <div class="countdown-display" id="timer">60</div>

        <!-- Control buttons -->
        <div class="controls">
            <button class="btn-start" id="startBtn">Start</button>
            <button class="btn-pause" id="pauseBtn" disabled>Pause</button>
            <button class="btn-reset" id="resetBtn">Reset</button>
        </div>
    </div>
```

```
<script>
    // Track the current time and timer interval ID
    let timeLeft = 60;
    let timerInterval = null;

    // Get DOM element references
    const timerDisplay = document.getElementById('timer');
    const startBtn = document.getElementById('startBtn');
    const pauseBtn = document.getElementById('pauseBtn');
    const resetBtn = document.getElementById('resetBtn');

    // Function to update the text showing on screen
    function updateDisplay() {
        timerDisplay.textContent = timeLeft;
    }

    // Starts or resumes the countdown
    function startTimer() {
        if (timerInterval !== null) return; // Prevent starting multiple intervals

        // Toggle button disabled states
        startBtn.disabled = true;
        pauseBtn.disabled = false;

        // Start repeating interval every 1000ms (1 second)
        timerInterval = setInterval(() => {
            timeLeft--;
            updateDisplay();

            // Check if time is up
            if (timeLeft <= 0) {
                clearInterval(timerInterval);
                timerInterval = null;
                startBtn.disabled = true;
                pauseBtn.disabled = true;
                timerDisplay.textContent = "0";
                timerDisplay.style.color = "#ef4444"; // Highlight in red when finished
            }
        }, 1000);
    }

    // Pauses the active countdown
    function pauseTimer() {
        if (timerInterval === null) return;

        clearInterval(timerInterval);
        timerInterval = null;

        // Toggle button states so user can resume
        startBtn.disabled = false;
        pauseBtn.disabled = true;
    }

    // Resets the timer to the starting condition
    function resetTimer() {
        clearInterval(timerInterval);
        timerInterval = null;
        timeLeft = 60;

        updateDisplay();
        timerDisplay.style.color = "#2563eb"; // Reset color to standard blue

        // Set buttons back to original state
        startBtn.disabled = false;
```

```
            pauseBtn.disabled = true;
        }

        // Bind events to buttons
        startBtn.addEventListener('click', startTimer);
        pauseBtn.addEventListener('click', pauseTimer);
        resetBtn.addEventListener('click', resetTimer);
    </script>

</body>
</html>
```

4. Verification & Testing

The project has been verified across Microsoft Edge, Google Chrome, and Mozilla Firefox. Performance validation was carried out by ensuring the setInterval logic does not duplicate when Start is repeatedly clicked. UI elements respond correctly to mouse hover effects, and button states transition appropriately (e.g., Pause disables when timer is not active).